- If you need to customize the services for every test, you can create a custom `WebApplicationFactory` by deriving from it and overriding the `ConfigureWebHost` method. You can place all your configuration in the custom factory and implement `IClassFixture<CustomWebApplicationFactory>` in your test classes instead of calling `WithWebHostBuilder()` in every test method.
- You can use the EF Core SQLite provider as an in-memory database to test code that depends on an EF Core database context. You configure the in-memory provider by creating a `SqliteConnection` with a `"DataSource=:memory:"` connection string. Create a `DbContextOptionsBuilder<>` object and call `UseSqlite()`, passing in the connection. Finally, pass `DbContextOptions<>` into an instance of your app's `DbContext`, and call `context.Database.EnsureCreated()` to prepare the in-memory database for use with EF Core.
- The SQLite in-memory database is maintained as long as there's an open `SqliteConnection`. When you open the connection manually, the database can be used with multiple `DbContext`s. If you don't call `Open()` on the connection, EF Core will close the connection (and delete the in-memory database) when the `DbContext` is disposed of.

---

1. `WebApplicationFactory` has many other methods you could override for other scenarios. For details, see https://learn.microsoft.com/aspnet/core/test/integration-tests.